

Team Setup Guide: Google Colab + GitHub UI + Central AWS S3 Storage

Repository: `https://github.com/ShubhamJain17r/Amazon-ML-Hackathon.git`

Official Team Workflow:

Our team uses **Colab's Native GitHub Integration** ("Save a copy in GitHub") for version control. **Zero git commands, zero terminal configurations, and no personal access tokens required!**

In AWS, we use **strictly Amazon S3** as our central cloud data store for all datasets and features.

1. The 3-Minute Team Onboarding (No Git Commands!)

Step 1: Open Google Colab & Connect to GitHub (1-Time)

1. Go to colab.research.google.com.
 2. Click **File** $\rightarrow$ **Open notebook**.
 3. Select the **GitHub** tab.
 4. Authorize Google Colab to connect to your GitHub account.
 5. In the repository search bar, select:
`text ShubhamJain17r/Amazon-ML-Hackathon`
 6. You will see all repository notebooks! Click your assigned notebook under `notebooks/<YourName>/` (or open a template to start fresh).
-

Step 2: Configure S3 Credentials in Colab Secrets

To read and write data to our central S3 database without exposing credentials in code:

1. In Colab's left sidebar, click the **Key icon (Secrets)**.
 2. Add these secrets (toggle **Notebook access** ON):
 - `AWS_ACCESS_KEY_ID`: `[Provided by Shubham]`
 - `AWS_SECRET_ACCESS_KEY`: `[Provided by Shubham]`
 - `AWS_DEFAULT_REGION`: `us-east-1`
 - `S3_BUCKET_NAME`: `[Provided by Shubham]`
-

Step 3: Universal Colab Notebook Header Cell

Paste and run this single cell at the very top of your Colab notebook:

```
# 1. Install cloud and performance libraries
!pip install -q boto3 s3fs pyarrow duckdb rapidfuzz lightgbm scikit-learn

import os
import boto3
import s3fs
import pandas as pd
from google.colab import userdata

# 2. Authenticate AWS S3 using Colab Secrets
os.environ["AWS_ACCESS_KEY_ID"] = userdata.get("AWS_ACCESS_KEY_ID")
os.environ["AWS_SECRET_ACCESS_KEY"] = userdata.get("AWS_SECRET_ACCESS_KEY")
os.environ["AWS_DEFAULT_REGION"] = userdata.get("AWS_DEFAULT_REGION", "us-east-1")
BUCKET = userdata.get("S3_BUCKET_NAME")

# 3. Verify S3 connection
fs = s3fs.S3FileSystem()
print(f"✅ S3 Central Store Connected! Bucket: s3://{BUCKET}/")
print(f"Available directories: {fs.ls(BUCKET)}")
```

2. Daily Workflow: Opening, Working, and Saving

How to Open Your Work Each Day

1. Open [Google Colab](#).
2. Click **File** → **Open notebook** → **GitHub** tab.
3. Select `ShubhamJain17r/Amazon-ML-Hackathon` and select your notebook under `notebooks/<YourName>/`.

How to Save Your Work (Commit Directly to GitHub)

When you finish a work session or complete an experiment:

1. In Colab's top menu, click: **File** → **Save a copy in GitHub**.
2. Set:
 - **Repository:** `ShubhamJain17r/Amazon-ML-Hackathon`
 - **Branch:** `main`
 - **File path:** `notebooks/<YourName>/<your_notebook_name>.ipynb`
 - **Commit message:** Type a clear description (e.g., `feat: added Hindi text normalization and token blocking`)
3. Click **OK**.

👉 **Done!** Your notebook is committed and pushed directly to GitHub. No merge conflicts, no terminal commands!

3. Accessing the Central S3 Cloud Data Store

Team members read and write directly to S3 via Parquet. Never save massive datasets inside notebooks!

Reading Filtered Data or Features

```
# Load feature Parquets prepared by Team A directly into pandas
train_df = pd.read_parquet(f"s3://{BUCKET}/features/sample_50k/train_candidates.parquet")
print(f"Loaded {len(train_df):,} rows from S3!")
```

Zero-RAM SQL Querying with DuckDB

```
import duckdb

con = duckdb.connect()
con.execute(f"""
    INSTALL httpfs; LOAD httpfs;
    SET s3_region='us-east-1';
    SET s3_access_key_id='{os.environ["AWS_ACCESS_KEY_ID"]}';
    SET s3_secret_access_key='{os.environ["AWS_SECRET_ACCESS_KEY"]}';
""")

# Query directly from S3 without consuming local RAM
high_prob_matches = con.execute(f"""
    SELECT source1_entity_id, candidate_entity_id, name_ratio
    FROM 's3://{BUCKET}/features/full/train_candidates.parquet'
    WHERE name_ratio > 0.85
    LIMIT 1000
""").df()
```

4. Using Antigravity AI for Code Generation & Progress Tracking

We use **Google Antigravity** as our primary AI pair programmer across the team.

How to Maintain Context Across Conversations in Antigravity

To ensure Antigravity understands your past progress, project architecture, and previous conversation context:

1. Provide the Conversation Reference Link:

At the start of any new Antigravity chat, reference the master project conversation:

text Context: This is part of the Amazon ML Challenge 2026 project. **Reference Conversation:** [conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a](https://www.conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a) **Repository:** [ShubhamJain17r/Amazon-ML-Hackathon](https://github.com/ShubhamJain17r/Amazon-ML-Hackathon) **Storage:** AWS S3 central cloud lake at `s3://<BUCKET>/` **Compute:** Google Colab

2. Use the Playbook Prompts:

Refer to docs/ai_prompt_playbook.md for pre-engineered, highly descriptive prompts for every stage of the competition.

3. Copy Code into Colab:

Antigravity outputs modular, complete code blocks that you can paste directly into your Colab notebook cells.